loader/social network. All these calls are asynchronous.
4. Each loader sends an HTTP call to an external API over the Internet. Once it's done returns the loaded data to the gatherer. Once all the loaders are finished, the gatherer returns all the collected data to its caller (the web server in this case).
5. The web server synchronously calls the sentiment analyzer, passing the raw data, and expect the summary in return. Alternatively, the web server could have called an orchestrator or the sentiment analyzer directly at point 2, and this step would be delegated.
6. Once the sentiment has been computed and passed back to the web server, it builds an HTTP response around it and sends it back to the client.

Latency can really be a problem because, say, you offered your customer a *Service Level Agreement* (SLA) where you committed to return 95% of your monthly calls within 750ms, but unfortunately a couple of those external services, during peaks of traffic, can take as long as 3s to answer.

To make things worse, your timeout for HTTP responses is set to 3.5s, meaning that you have to return an answer to the client within 3.5 seconds, otherwise your load balancer will kill the call; I can imagine you are thinking it would be enough to adjust the timeout, but suppose you can't change this timeout otherwise you couldn't support the traffic load, given the resources you have available. So, assuming you take around 250ms to process data for a single source, if it takes 3 seconds to get that data, considering the time to